

# Programming Paradigm

Programming paradigm means style or the standard way of writing a program

Programming paradigms are the various standard approaches to write a program

When we are choosing a programming language for writing the we need to make sure it is supporting the programming paradigm approach.

Why we need the programming Paradigm:

"Any fool can write code that a computer can understand"  
Good programmers write code that humans can understand"

Programming without understanding programming paradigm

Increase complexity

less Readability

less structure

Hard to test

Hard to change

Hard to maintain

\* Access specifiers are keywords used to set the access level or visibility of classes, methods, and other constructs.



## Common types of programming paradigms

- Imperative programming
  1. Procedure-oriented programming (POP)
  2. Object-oriented programming (OOP)
- Declarative programming
  1. functional programming
  2. logic programming

## Imperative Programming

Imperative programming tells the computer how to do the task by giving a set of command or instruction in a particular order.

### • Procedure-oriented programming

In POP the complete program is divided into multiple small parts called function or subroutine.

It focuses on functions as well as the sequence of action to be done.

Procedure programming follows a Top-down approach.

Don't have access specifiers.

- \* Access specifiers are keywords used to set the access level or visibility of classes, methods and other members.

They control which parts of a program an access certain members.

In the Procedural programming, most of the functions uses global data.

POP does not offer any method of hiding data. Thus it is less secure as compared to the object oriented programming.

Some Common examples of Procedural programming are - C, Fortran, VB and Pascal.

## • Object-oriented programming

In OOP the program is divided into multiple small parts called object. **In OOP object is a fundamental unit that represents an instance of a class.**

It focuses on the data rather than functions or procedures.

Object programming follows a Bottom-Up approach.

**Have access specifiers like public, private, protected etc.**

An OOP we can control the access of data using access specifiers.

And we can do it by using access specifiers.



Hiding data is possible with object oriented programming due to the abstraction. Thus, it is more secure than the procedural programming.

Principle

Concepts of object-oriented programming

Objects

Classes

Encapsulation

Inheritance

Abstraction

Polymorphism

### • Class

Class is a blueprint of an object.

Class is a description of object's property set and set of operation.

It is a user-defined data type like structure but it contains data members as well as member functions.

Creating Class is as good as defining a new data type.

Class is a means to achieve encapsulation.

The wrapping up of data and functions into a single unit (called class) is known as encapsulation.

The data is not accessible to the outside world, and only those functions which are wrapped in the class can access it.

And we can do it by using access specifiers.



## • Object

Object is a run time entity

Object can be defined as creating a variable with class data type

Object is slightly different from variable as variable contain only single data value but object contain multiple value.

Objects take up space in the memory and have an associated address.

## Inheritance

Inheritance is the process by which objects of one class acquire the properties of objects of another class.

## Abstraction

Data abstraction is one of the most essential and important features of object-oriented programming in C++.

Abstraction means displaying only essential information and hiding the details.

Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

## Polymorphism

Polymorphism means the ability to take more than one form.

An operation may exhibit different behaviors in different instances. The behavior depends upon the types of data used in the operation.

We can achieve polymorphism by two types -

- Overloading
- Overriding

### Overloading

Using single function name to perform different types of tasks.

### Overriding

Overriding is needed when derived class function has to do some different job than the base class function.



## Introduction to C++

C++ is an object-oriented programming language.

It was developed by Bjarne Stroustrup at AT & T Bell Laboratories in USA in the early 1980's.

Bjarne was inspired by Simula67 & it is the first object oriented language.

Bjarne combined the oops features of Simula67 with the C language and made a new language called "C with classes".

In 1983, the name was changed to C++. The idea of C++ comes from the C increment operator ++, thereby suggesting that C++ is an incremented version of C.

C++ is a superset of C.

C++ is a Middle level language.



## features of C++

C++ is an object-oriented programming language.

- ★ But C++ is not purely object-oriented programming language. C++ is flexible and compatible with other paradigms make it a multi-paradigm language.

Means you can write C++ code without using classes or objects at all.

C++ is case sensitive.

C++ is a compiled language.

# Compiler / Interpreter

## Compiler

- A compiler is a type of software that translates code written in a high-level programming language into machine code, which is then executable by the computer's hardware.
- Compiler translates entire source code into machine code before execution.
- Errors detected at compile time, before execution.

## Interpreter

- A Interpreter is a type of software that directly executes the high-level programming language.
- It translates and executes code simultaneously line by line or statement by statement.
- Errors detected at runtime during execution.